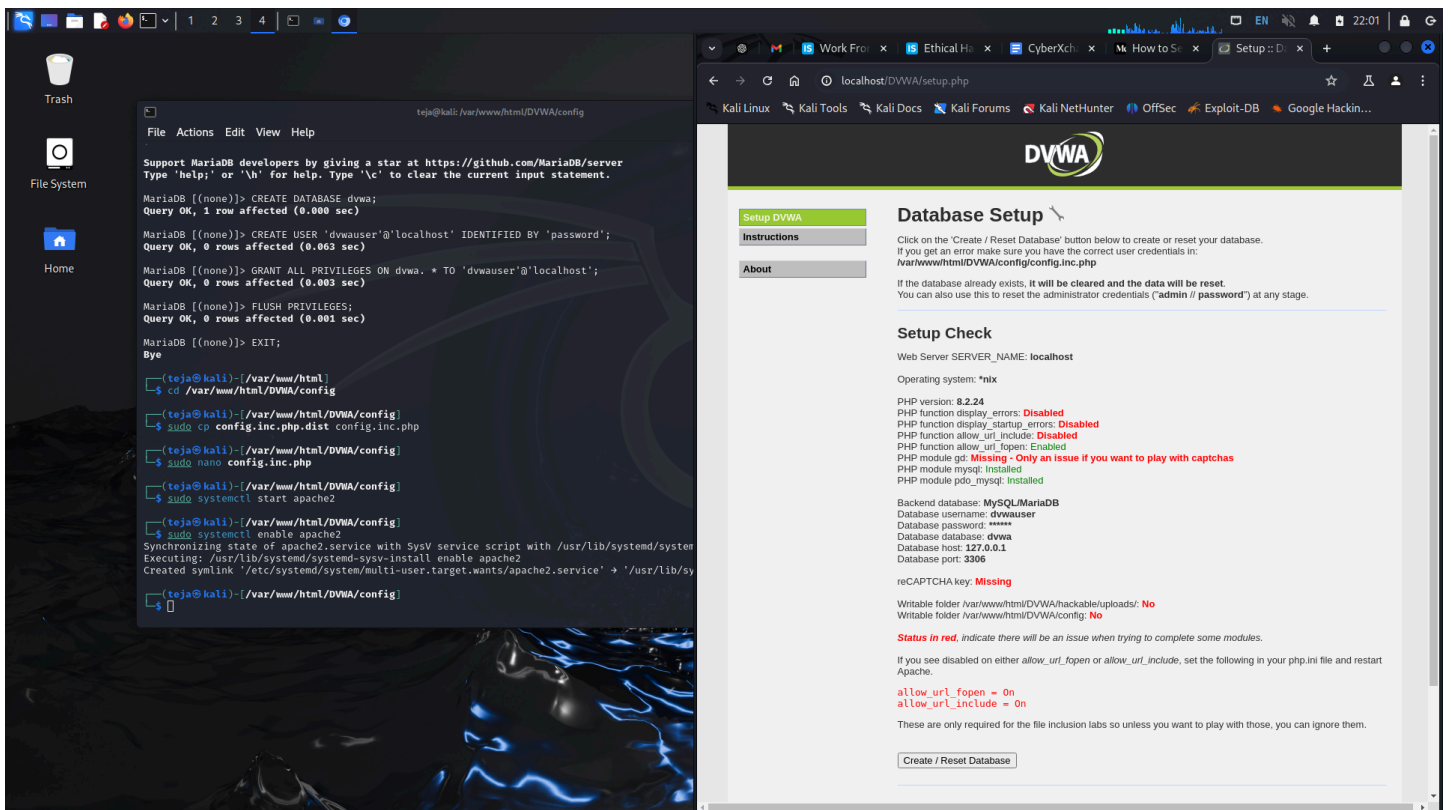
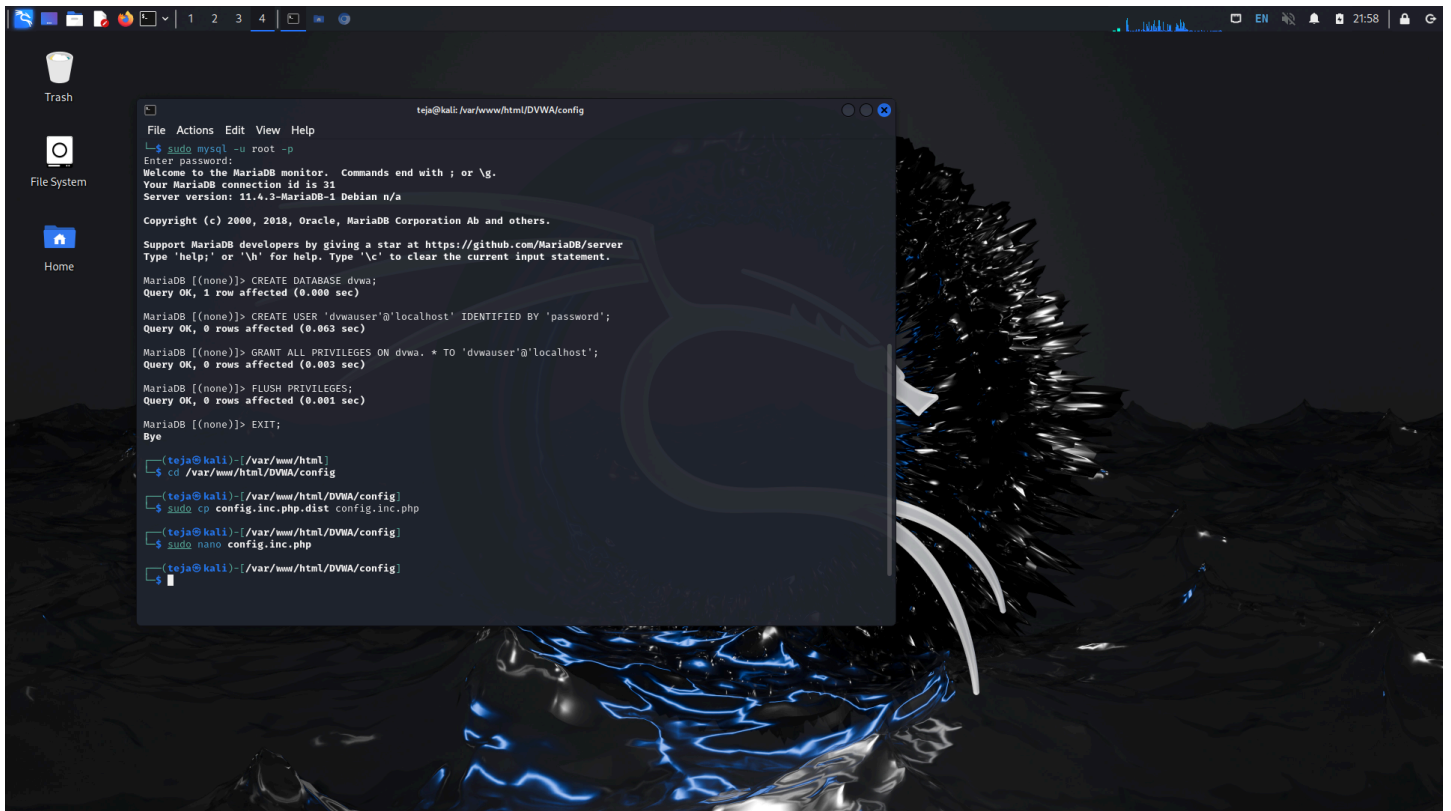


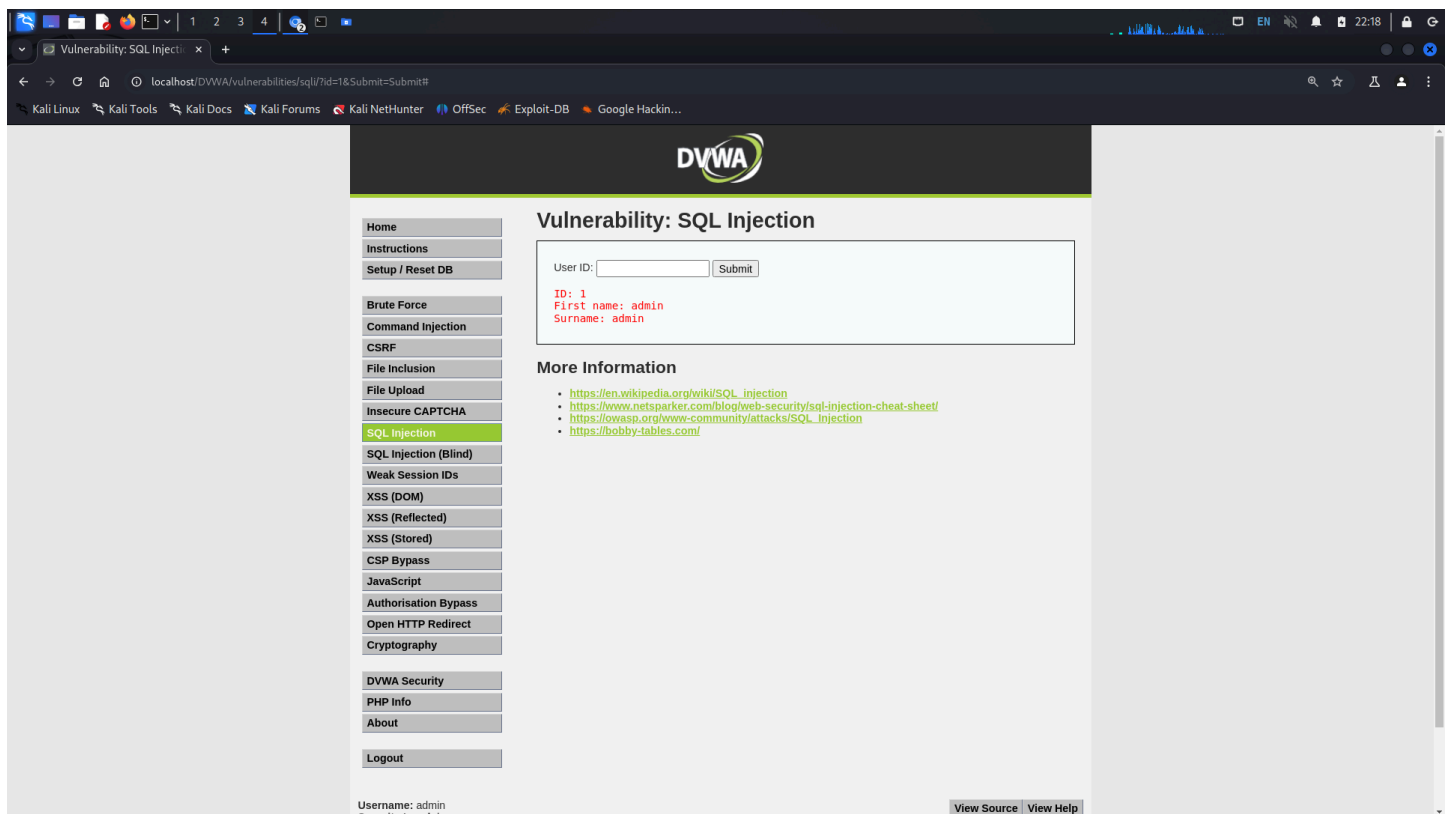
Installation

1) DVWA Setup

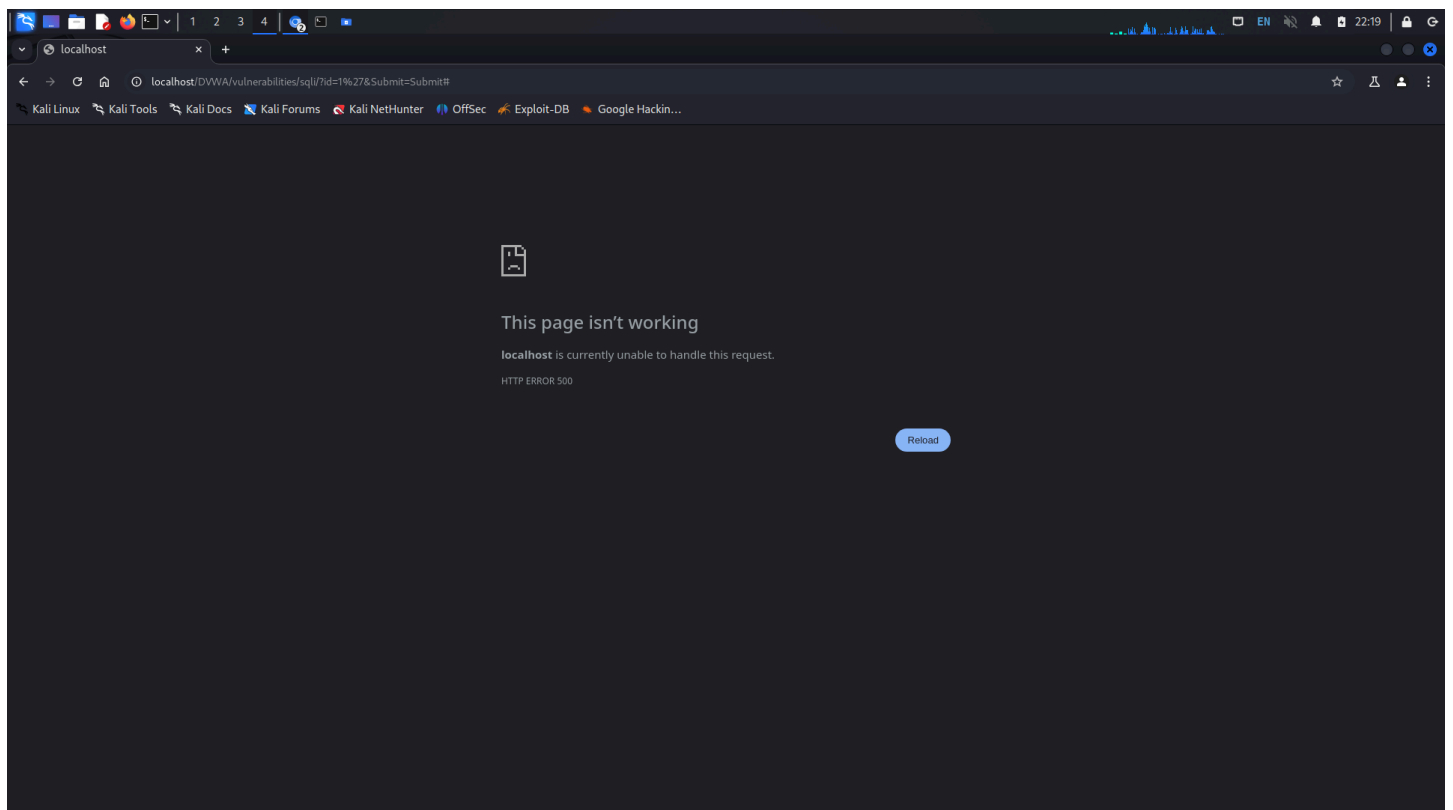


Vulnerability Testing

2) SQL Injection

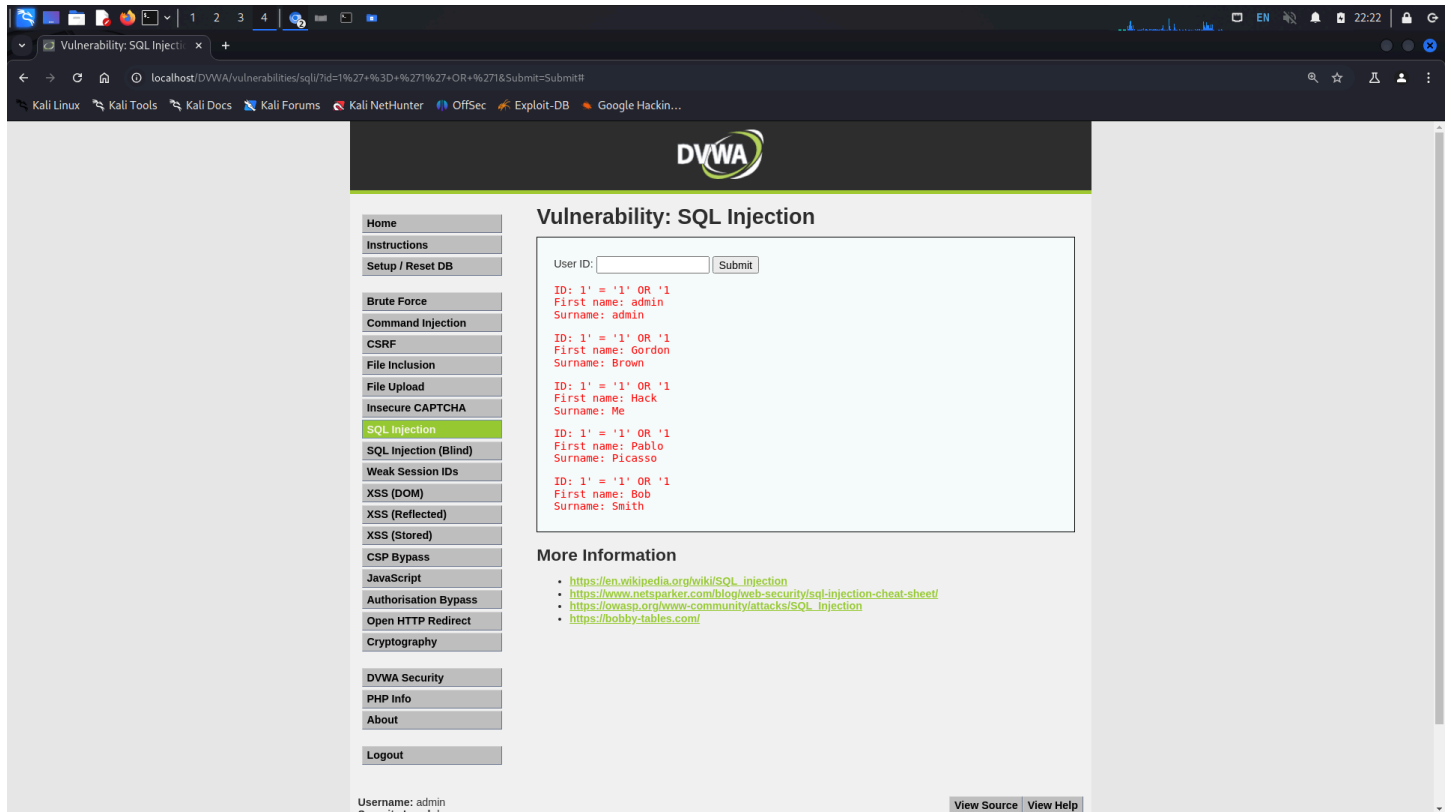


In the above screenshot, I submitted USER ID as 1, casually to check the output



Whereas, upon submitting 1' it resulted in the above HTTP ERROR 500

It confirms that the web application is improperly handling the input and is interfering with the SQL syntax



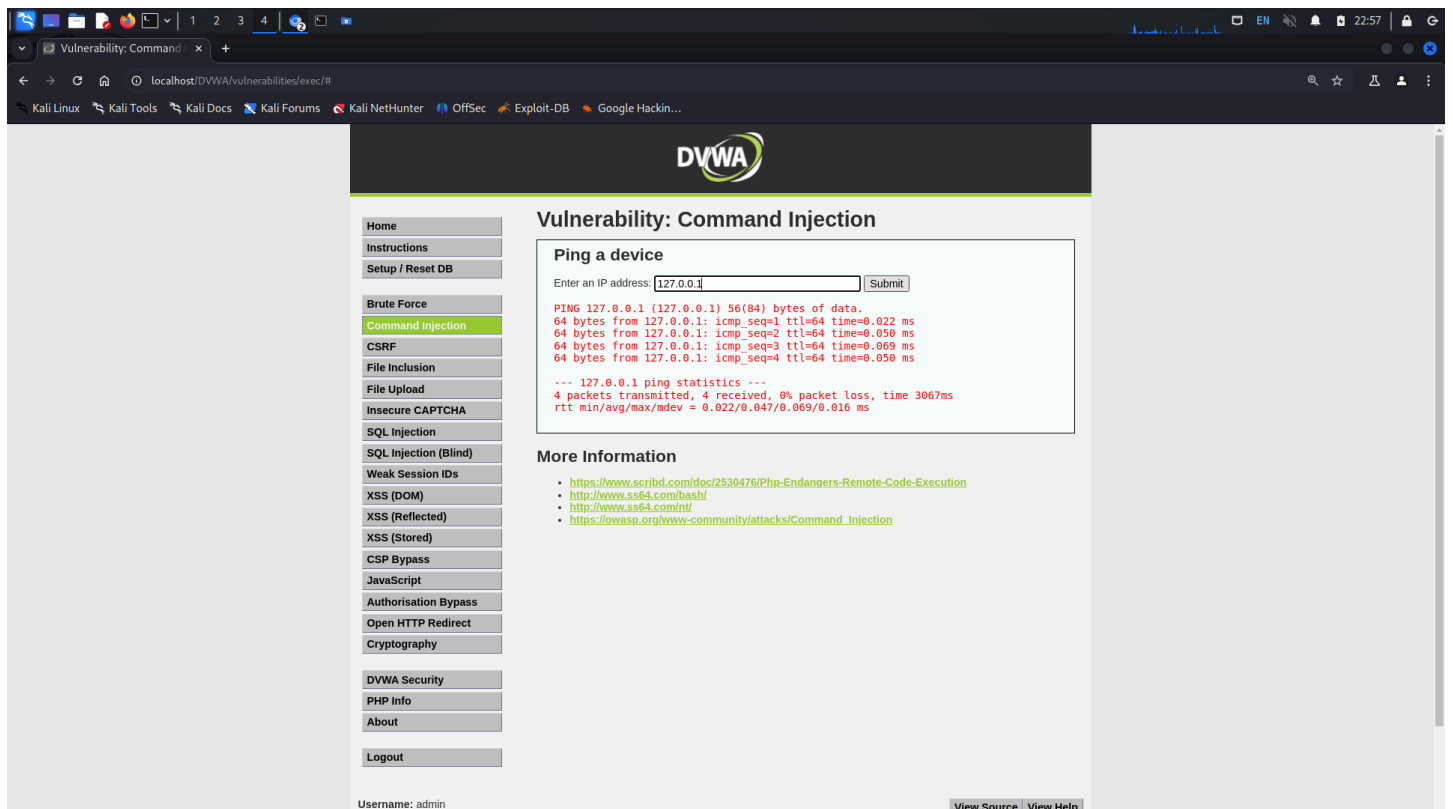
Further, I went on testing by submitting `1' = '1' OR '1` as USER ID

My assumption was correct

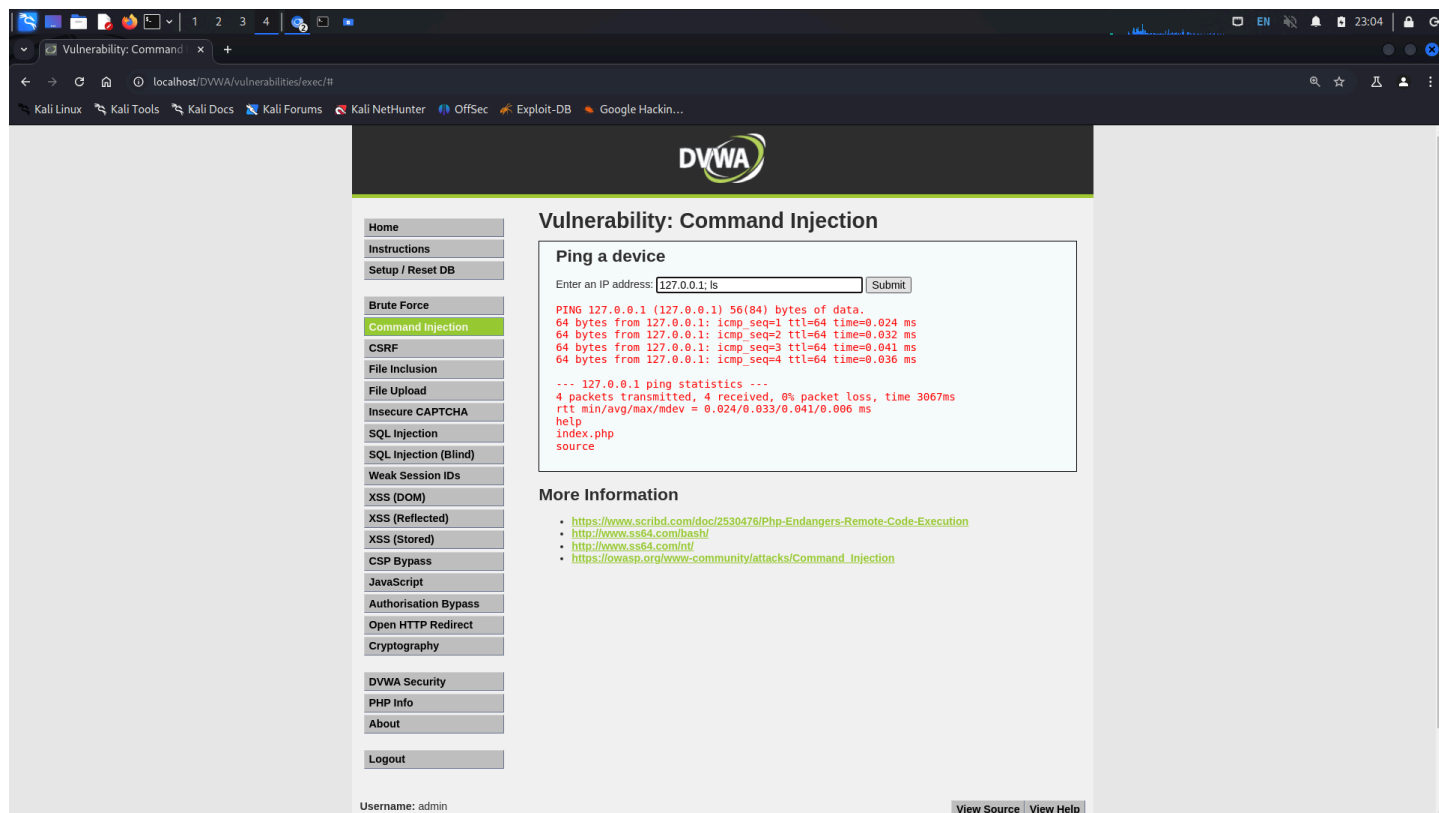
The input interfered with the backend SQL Query

Because the condition `'1' = '1'` is always true, the manipulated backend query retrieved all the records in the table for me

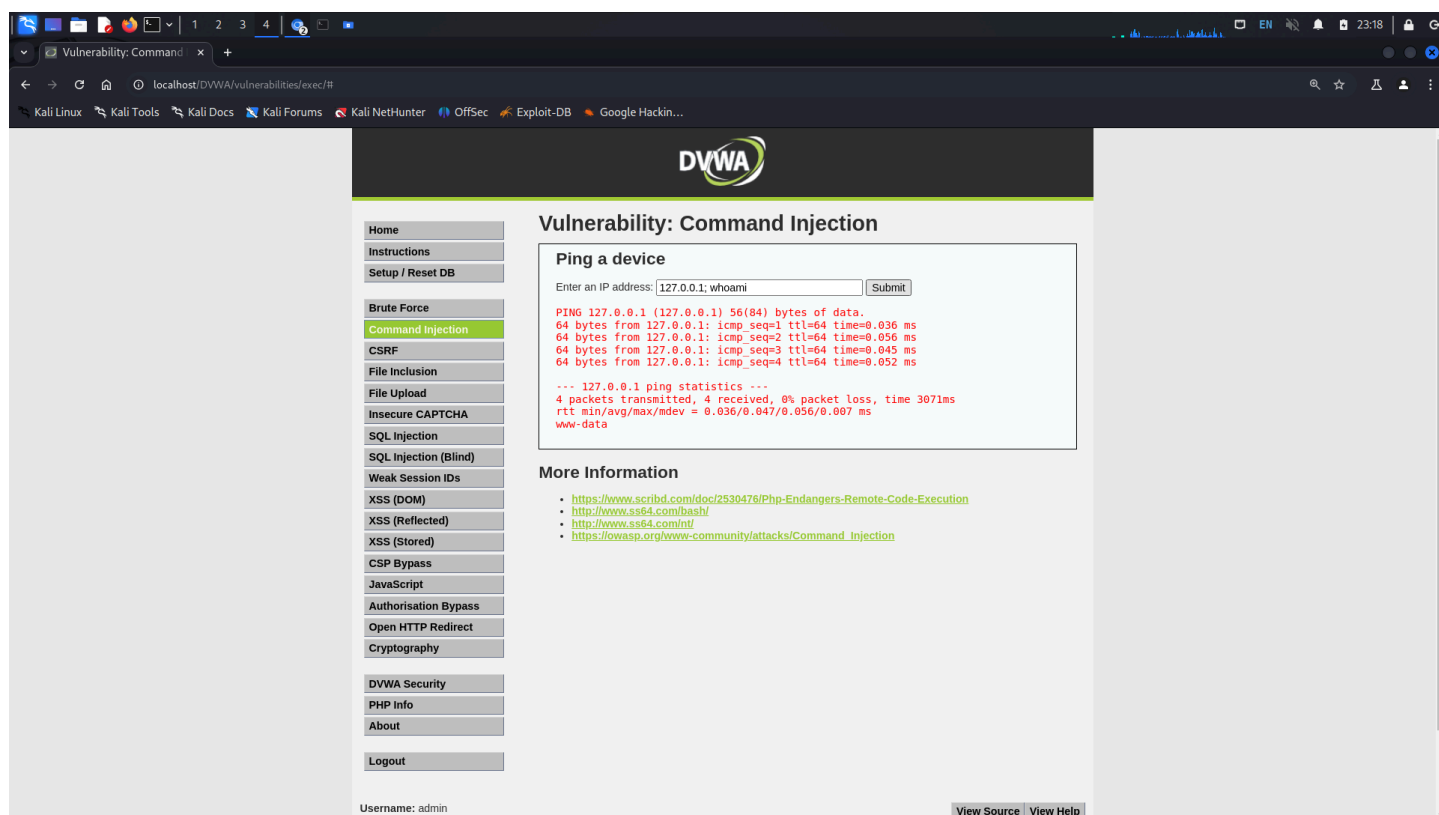
3) Command Injection



First, I casually submitted my loopback address (127.0.0.1) as input to check if it accepts user input and executes the ping command. It executed the ping command successfully and the output was as shown in the above image.

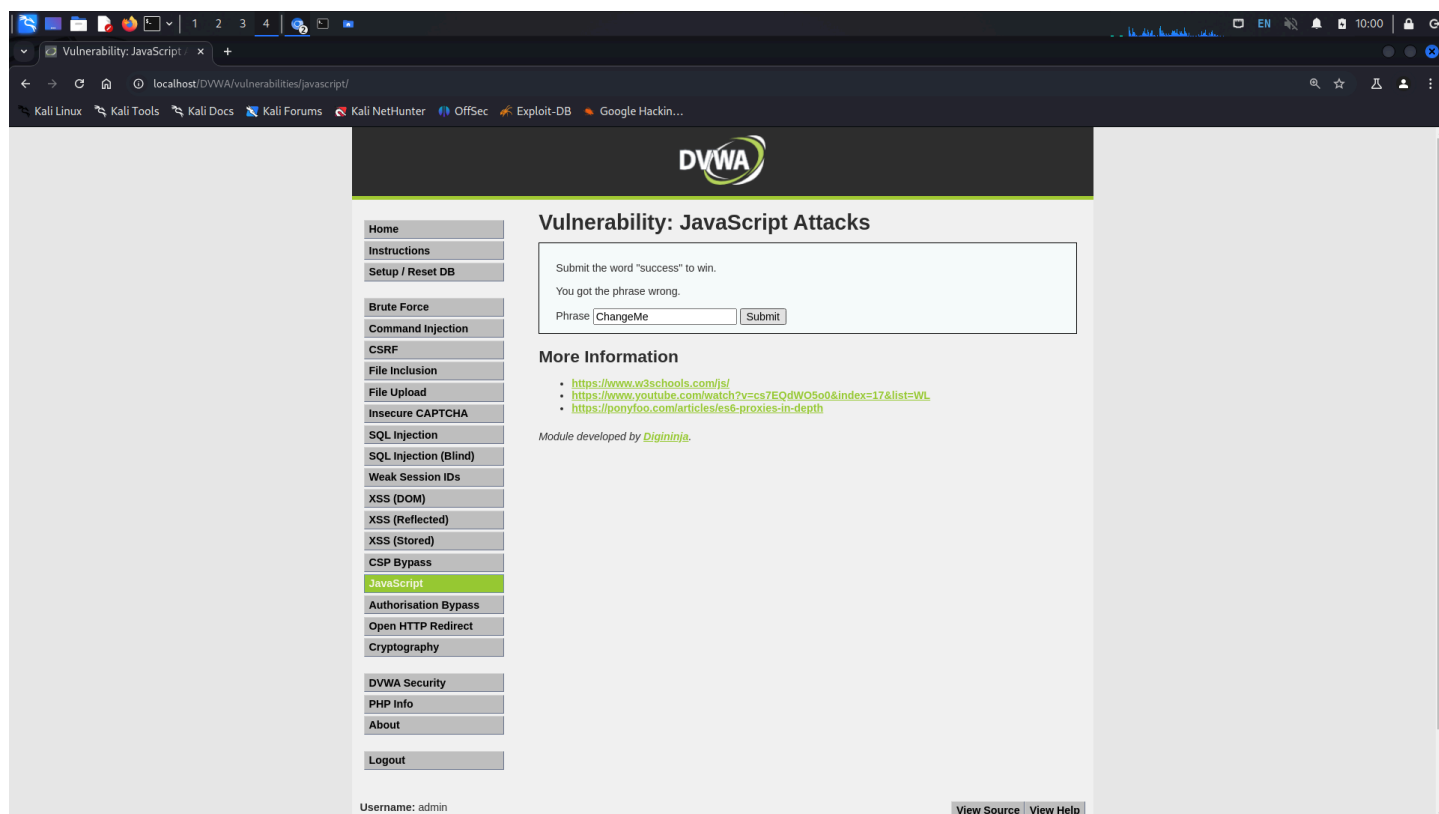


Next, I appended the **ls** command to the loopback address using the semicolon ;
In addition to the the output of the ping, it also gave me the list of files present in its server
The output can be seen in the above image

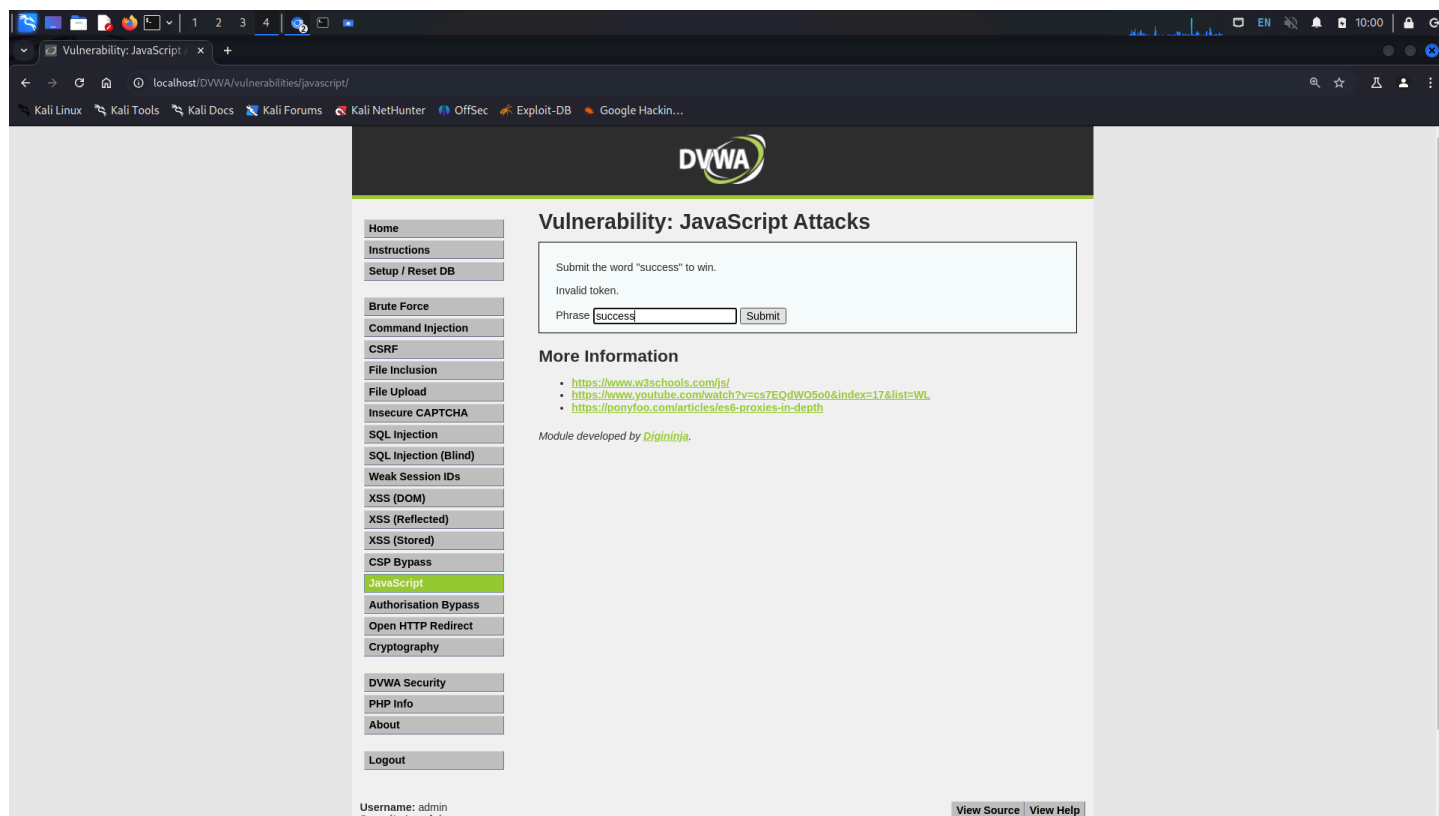


Further, out of intuition, I appended the **whoami** command to enquire the user under which the application's server is running
The output was **www-data**
This can be seen in the above image
Clearly, we can say that it is prone to command injection

3) JavaScript Attack



The default value in the input was **ChangeMe**, upon submitting it, I got a message that the phrase was wrong. This is visible in the above screenshot



Because, I was prompted to submit the word "success" to win, I tried submitting the phrase success. However, the submission resulted in an Invalid token.

1234

Vulnerability: JavaScript

localhost/DVWA/vulnerabilities/javascript/

Kali LinuxKali ToolsKali DocsKali ForumsKali NetHunterOffSecExploit-DBGoogle Hackin...

DVWA

HomeInstructionsSetup / Reset DBBrute ForceCommand InjectionCSRFFile InclusionFile UploadInsecure CAPTCHASQL InjectionSQL Injection (Blind)Weak Session IDsXSS (DOM)XSS (Reflected)XSS (Stored)CSP BypassJavaScriptAuthorisation BypassOpen HTTP RedirectCryptographyDVWA SecurityPHP InfoAboutLogout

Vulnerability: JavaScript Attacks

Submit the word "success" to win.

You got the phrase wrong.

Phrase

More Information

- <https://www.w3schools.com/js/>
- <https://www.youtube.com/watch?v=cs7EQdWO5o0&index=17&list=WL>
- <https://ponyfoo.com/articles/es6-proxies-in-depth>

Module developed by [Digininja](#).

ElementsConsoleSourcesNetworkPerformance

topFilterDefault levelsNo Issues

```
> const phrase = "success";
const rot13_trans_phrase = rot13(phrase);
const token = md5(rot13_trans_phrase);

console.log("rot13 transformed phrase: ", rot13_trans_phrase);
console.log("generated token: ", token);
rot13 transformed phrase:  fhppirff
generated token:  38581812d435834ebf84ebcc2c6424d6
```

underlined

Username: admin

View SourceView Help

I came up with an idea to obtain the token value of the phrase "success", so that I get the output correct upon submission I passed the success phrase through the rot13 function and then this transformed phrase into md5 hash function and printed both of them in the console

The generated token is something like "385.....d6"

Vulnerability: JavaScript Attacks

Submit the word "success" to win.

You got the phrase wrong.

Phrase

More Information

- <https://www.w3schools.com/js/>
- <https://www.youtube.com/watch?v=cs7EQdWO5o0&index=17&list=WL>
- <https://ponyfoo.com/articles/es6-proxies-in-depth>

Module developed by [Digininja](#).

Username: admin
Security Level: low

[View Source](#) [View Help](#)

Using the generated token, I updated the current token value of “8b7...09” to “385...d6”

Vulnerability: JavaScript Attacks

Submit the word "success" to win.

Well done!

Phrase

More Information

- <https://www.w3schools.com/js/>
- <https://www.youtube.com/watch?v=cs7EQdWO5o0&index=17&list=WL>
- <https://ponyfoo.com/articles/es6-proxies-in-depth>

Module developed by [Digininja](#).

Username: admin
Security Level: low

[View Source](#) [View Help](#)

Now, after updating the token value, when I submitted the phrase “success”, I got the message “Well done!”

Transparency in the logic behind encrypting the user input and manipulating it to obtain the desired output, makes this code portion vulnerable and easy to exploit

4) File Upload Vulnerability

The screenshot shows the DVWA File Upload page. The left sidebar contains a menu with options like Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload (highlighted), Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript, Authorisation Bypass, Open HTTP Redirect, Cryptography, DVWA Security, PHP Info, About, and Logout. The main content area displays the following error messages:

- Incorrect folder permissions: /var/www/html/DVWA/hackable/uploads/ Folder is not writable.
- The PHP module GD is not installed.

Below the errors, there is a section for uploading an image. It says "Choose an image to upload:" and shows a file named "Screenshot...2_19_25.png" selected. The "Upload" button is visible, but the message "Your image was not uploaded." is displayed below it.

The "More Information" section provides links to external resources:

- https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- <https://www.acunetix.com/websecurity/upload-forms-threat/>

On the right, the browser's developer tools are open, showing the source code of the page. The code is a PHP file named "upload.php". It contains a form with an input field for the file name and a submit button labeled "Upload". The code also includes a warning message about folder permissions and a message indicating that the image was not uploaded.

At first, I uploaded a 900 kb png file. It failed. Upon html code inspection, I realised that the `MAX_FILE_SIZE` is set to 100 kb only. However, even after uploading a 60 kb, it failed. This was because the uploads folder permission initially was **not writable**

The screenshot shows the DVWA File Upload page after the folder permissions have been changed. The left sidebar is the same as in the previous screenshot. The main content area displays the following error messages:

- The PHP module GD is not installed.

Below the errors, there is a section for uploading an image. It says "Choose an image to upload:" and shows a file named "Screenshot...2_19_25.png" selected. The "Upload" button is visible, but the message "Your image was not uploaded." is displayed below it.

The "More Information" section provides links to external resources:

- https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
- <https://www.acunetix.com/websecurity/upload-forms-threat/>

On the right, the browser's developer tools are open, showing the source code of the page. The code is a PHP file named "upload.php". It contains a form with an input field for the file name and a submit button labeled "Upload". The code also includes a warning message about folder permissions and a message indicating that the image was not uploaded.

In the background, a terminal window shows the following commands and output:

```

teja@kali: ~
└─(teja@kali)~
└─$ chmod 777 /var/www/html/DVWA/hackable/uploads/
chmod: changing permissions of '/var/www/html/DVWA/hackable/uploads/': Operation not permitted
└─$ sudo chmod 777 /var/www/html/DVWA/hackable/uploads/
[sudo] password for teja:
└─(teja@kali)~
└─$
  
```

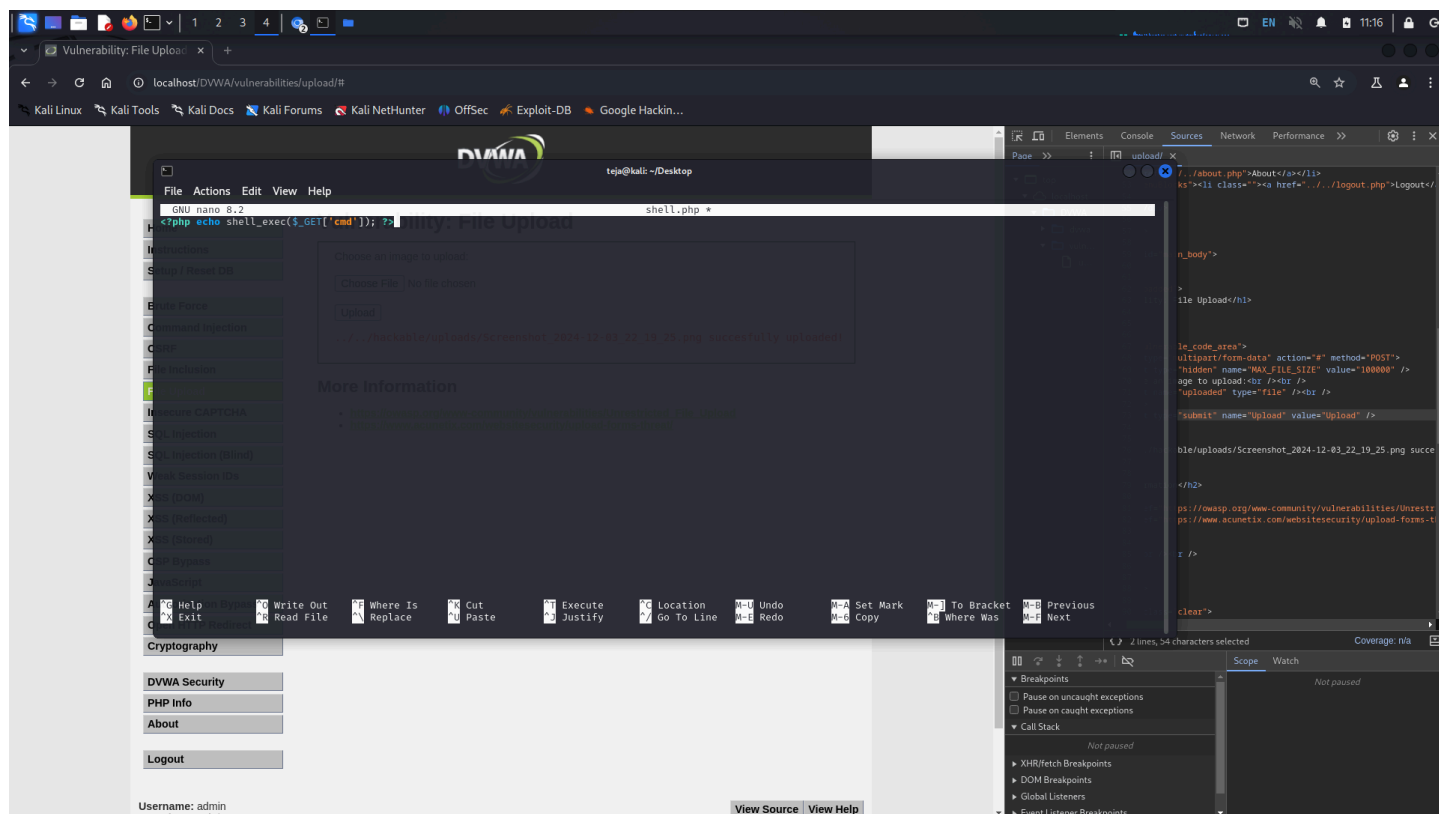
Immediately, I changed the folder permission to 777 using `chmod`. This helped me get rid of the incorrect folder permission issue.

The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. The 'File Upload' tab is selected in the left sidebar. A message at the top states: "The PHP module GD is not installed." Below this, a file upload form is shown with a 'Choose File' button and an 'Upload' button. A message below the form indicates: ".../hackable/uploads/Screenshot_2024-12-03_22_19_25.png succesfully uploaded!". The 'More Information' section lists links to OJpwp.org and Acunetix.com. The right sidebar shows the 'Sources' tab with the HTML source code of the page, highlighting the upload form and the success message.

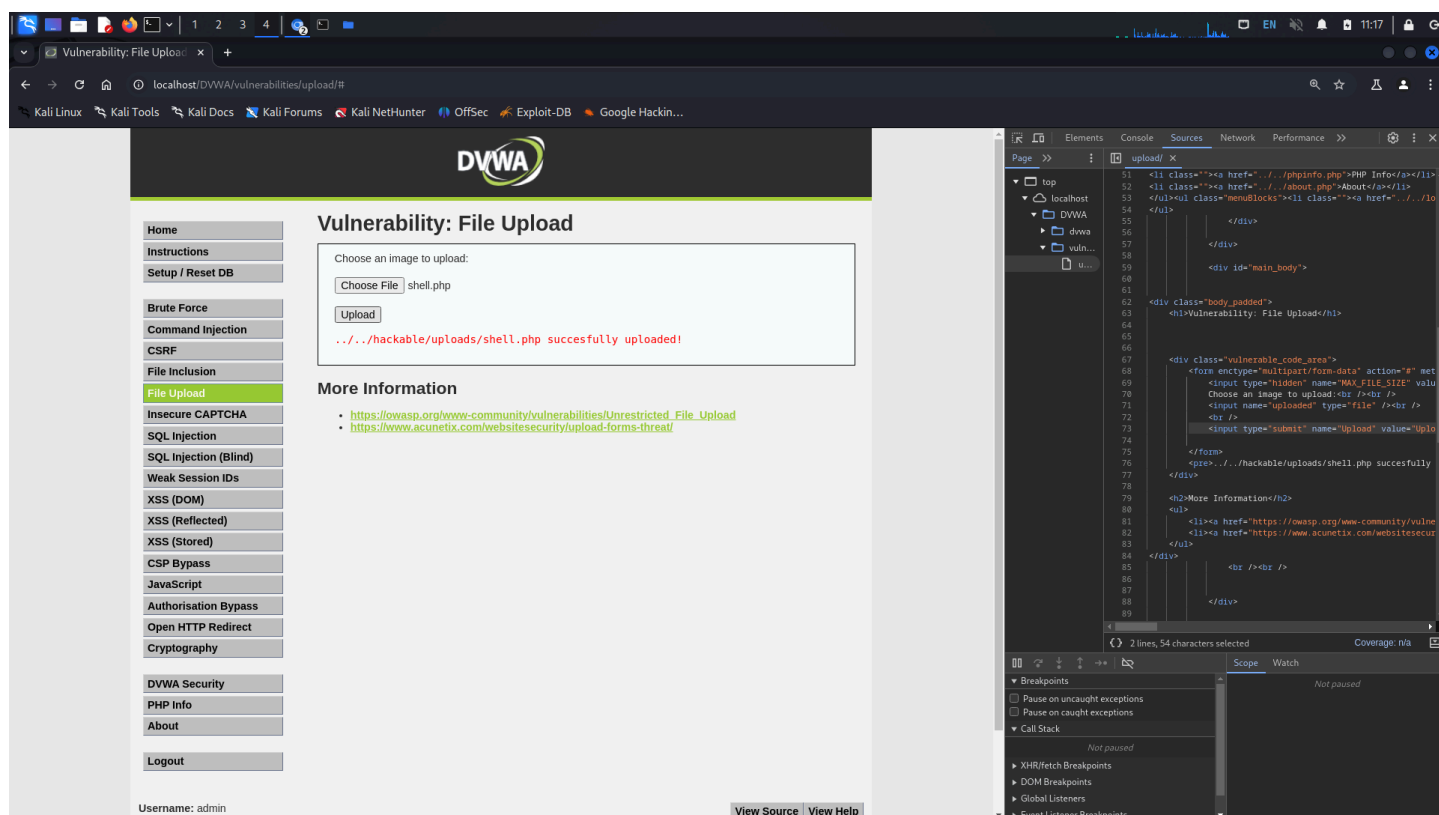
And the 60 kb image that I previously selected got uploaded successfully

The screenshot shows a terminal window on a Kali Linux system. The user is in the directory /etc/php/8.2/mods-available. The terminal output shows the installation of php-gd, including the download of the package, the extraction of the files, and the creation of the config file. The user then runs the command 'sudo systemctl restart apache2' to restart the web server. The right sidebar shows the 'Sources' tab with the HTML source code of the page, highlighting the upload form and the success message.

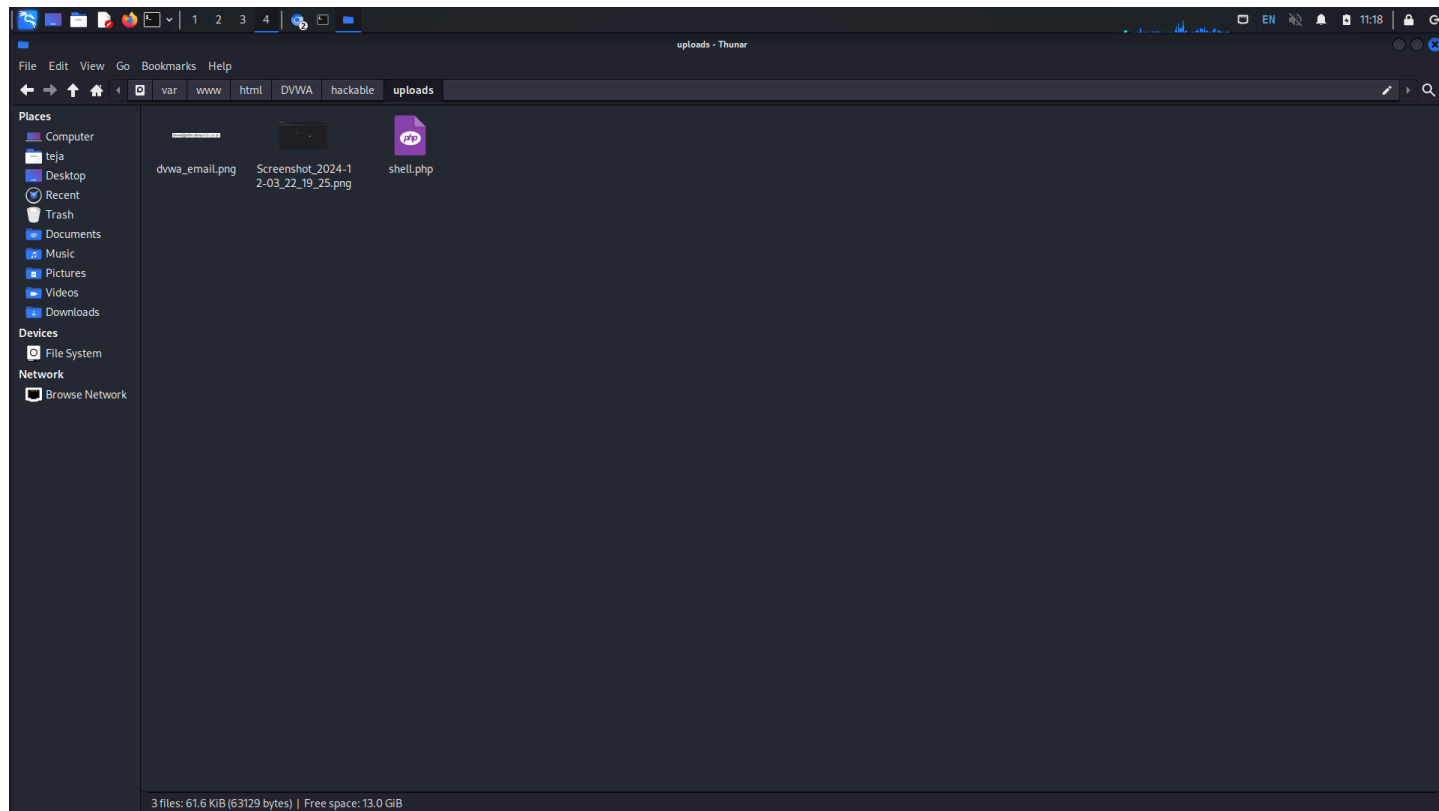
Further, I installed php-gd to ensure image processing and validation takes place without any issue
Then restarted my apache2 server to check if the php gd issue is resolved



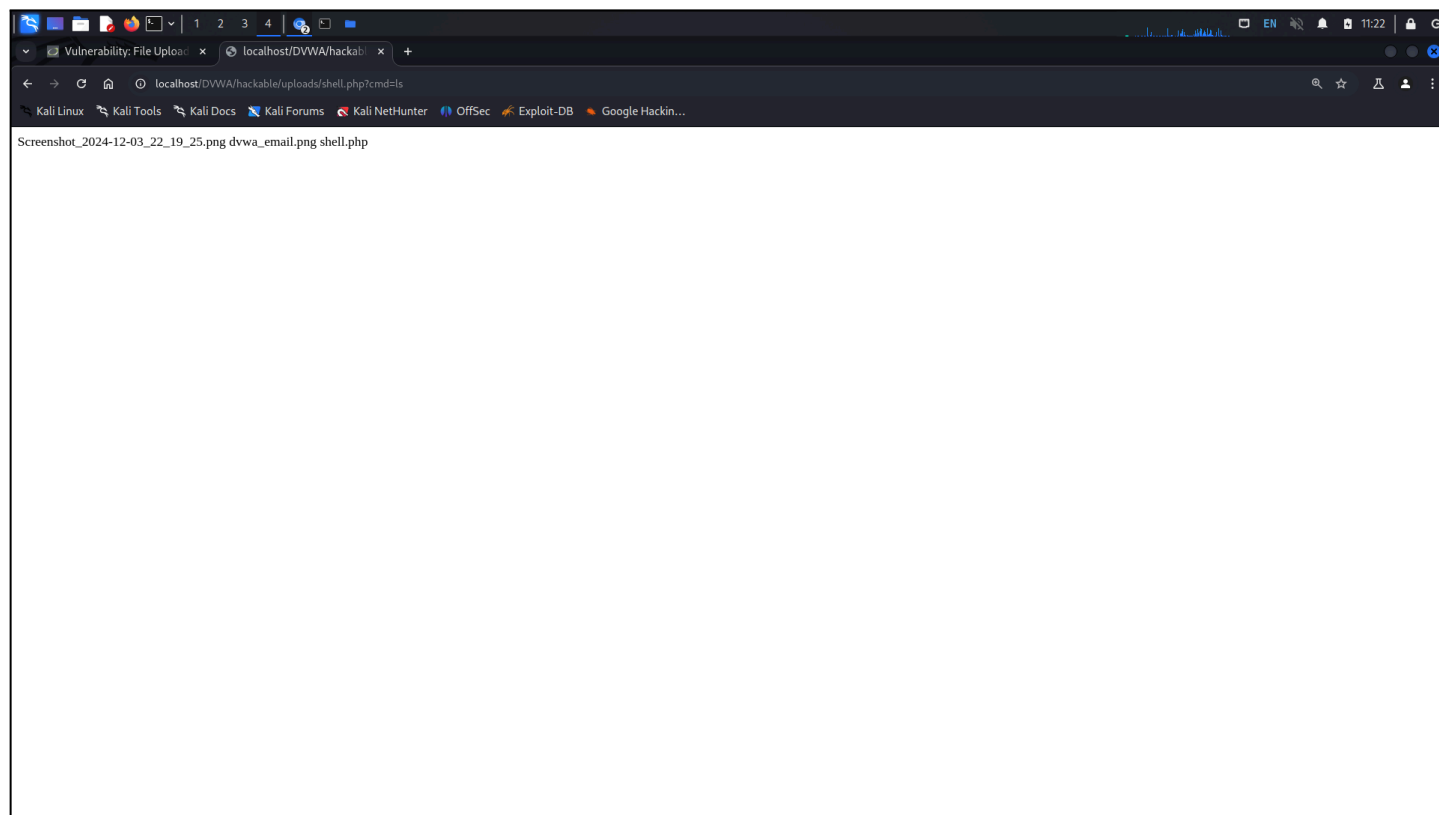
Next, using nano, I created a shell.php file on my desktop containing a potentially malicious code that can perform unrestricted command execution



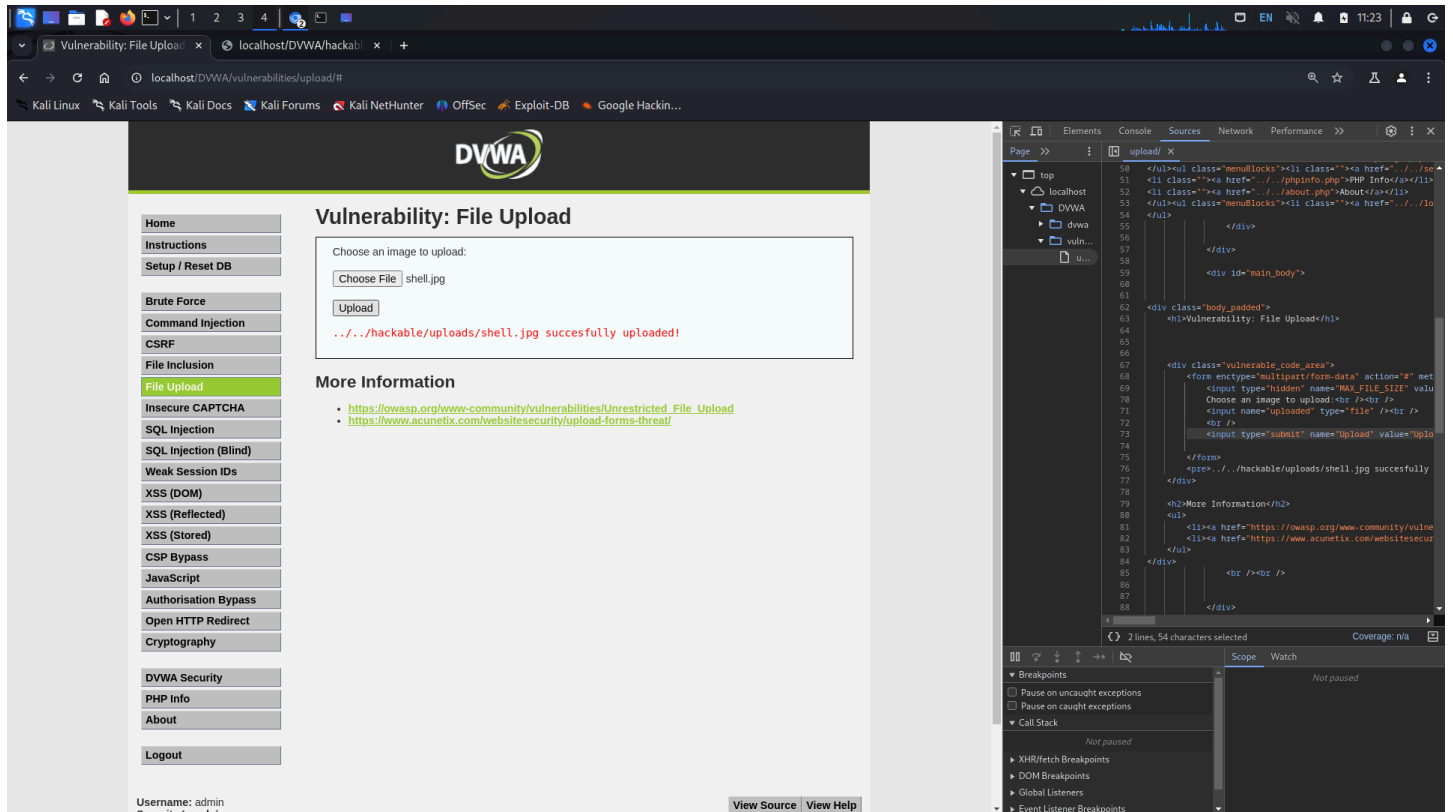
Upon uploading this shell.php, the submission was successful
I noticed that there were no restrictions on the image file format (png/ jpg/ heic etc.), which might not be a significant issue
However, the code is also not verifying whether the file being uploaded is really an image file or not



The files were successfully uploaded and stored in the location `/var/www/html/DVWA/hackable/uploads`. It can be seen in the above image.



When I tried accessing this file and executed `cmd=ls`, I could successfully retrieve the list of files present in the folder.



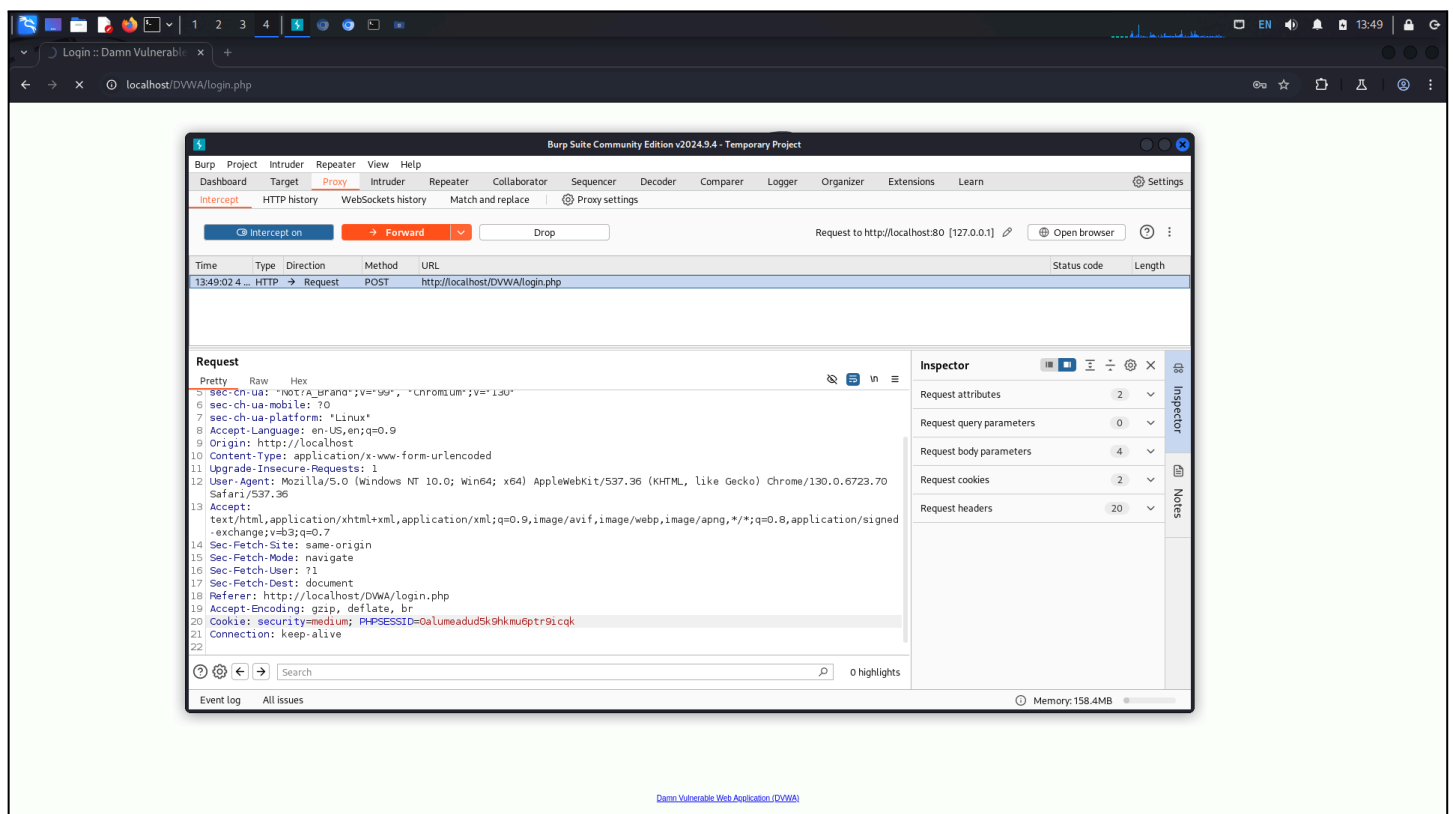
Furthermore, I went on to rename the php file to jpg and uploaded it

The upload was successful

Which means, the internal contents of a file are not being processed and verified

Any malicious attacker can easily upload a malicious code in with a jpg extension and perform remote code execution using it

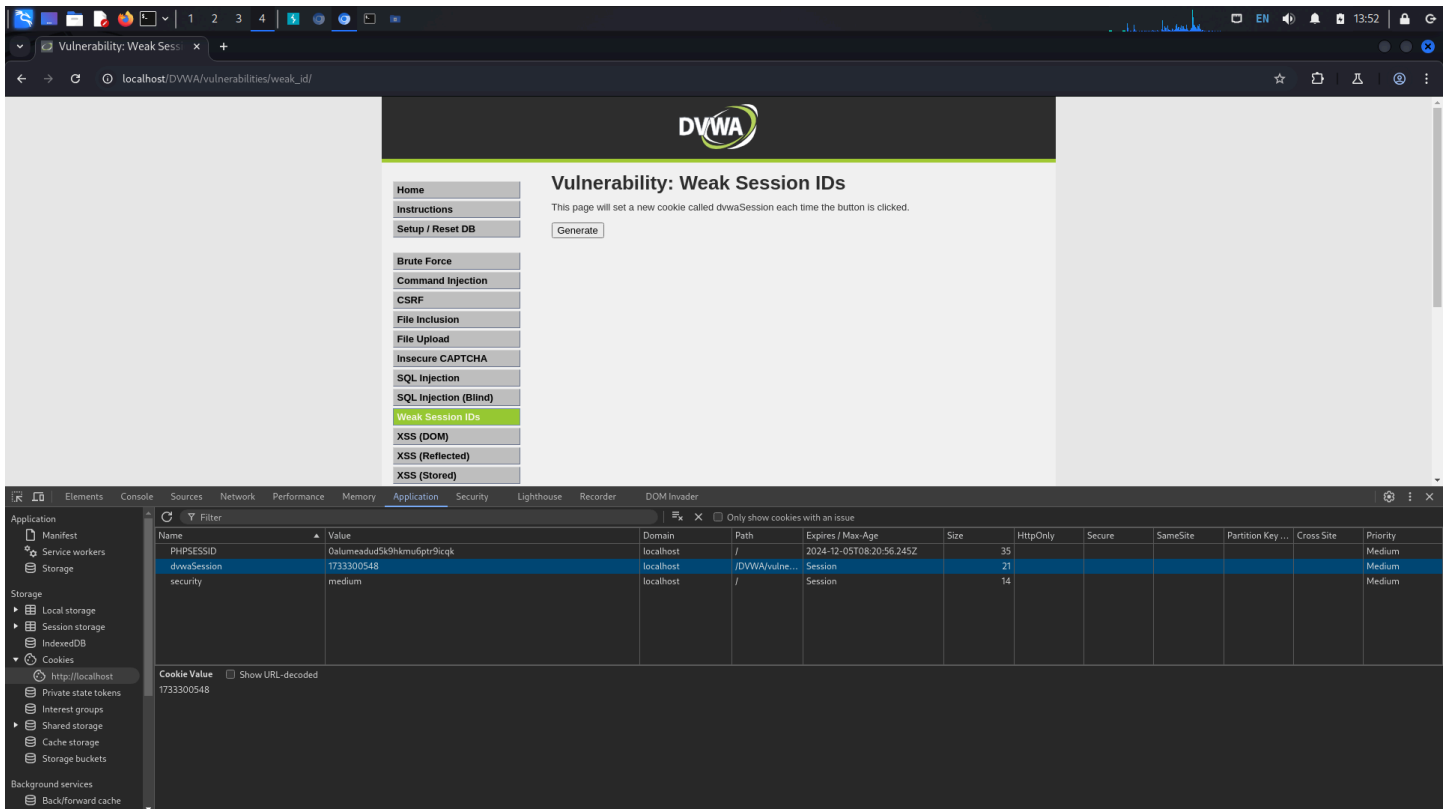
5) Weak Session ID



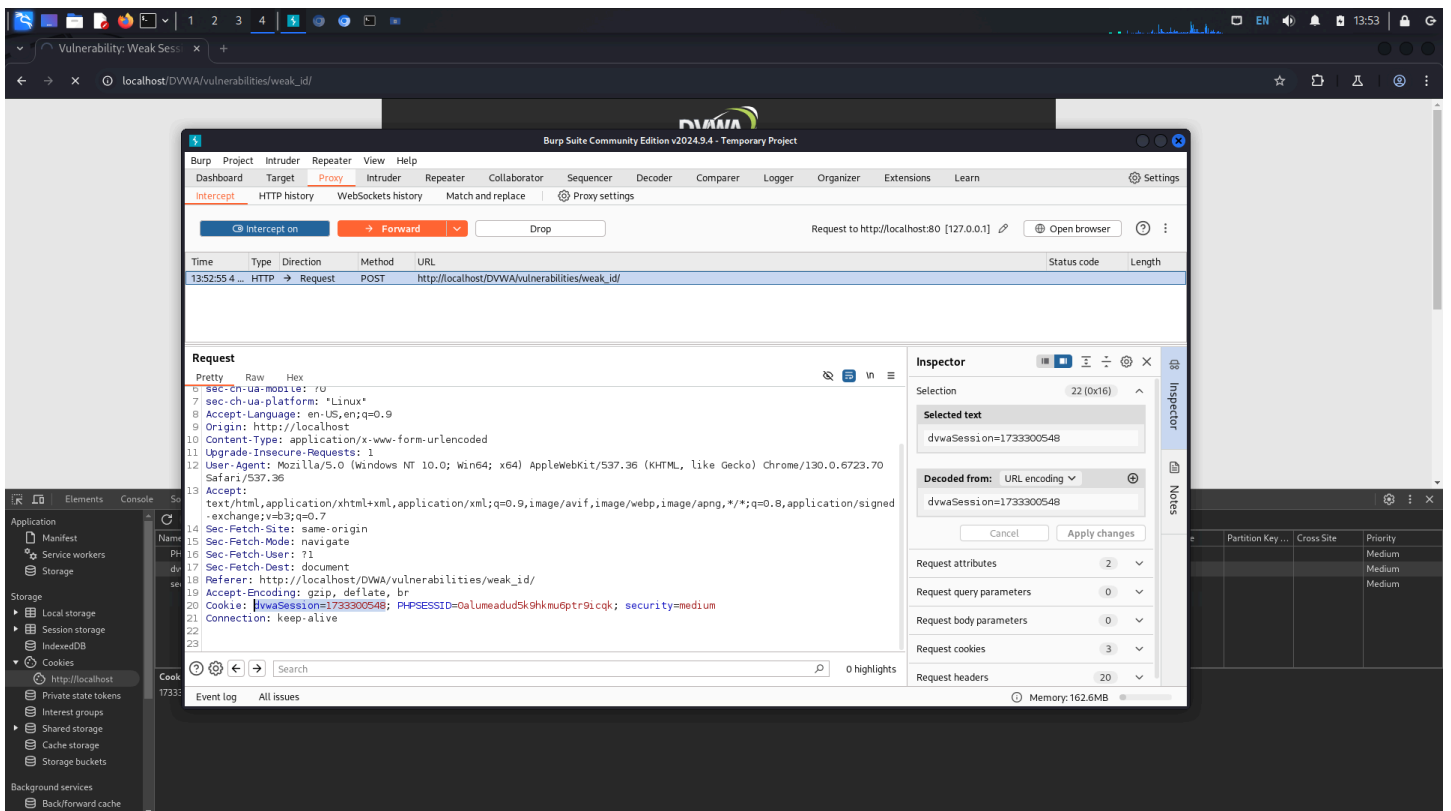
To test the weak session ID vulnerability, I utilized burp suite proxy to intercept and capture details of each request

The cookie values while intercepting are clearly visible

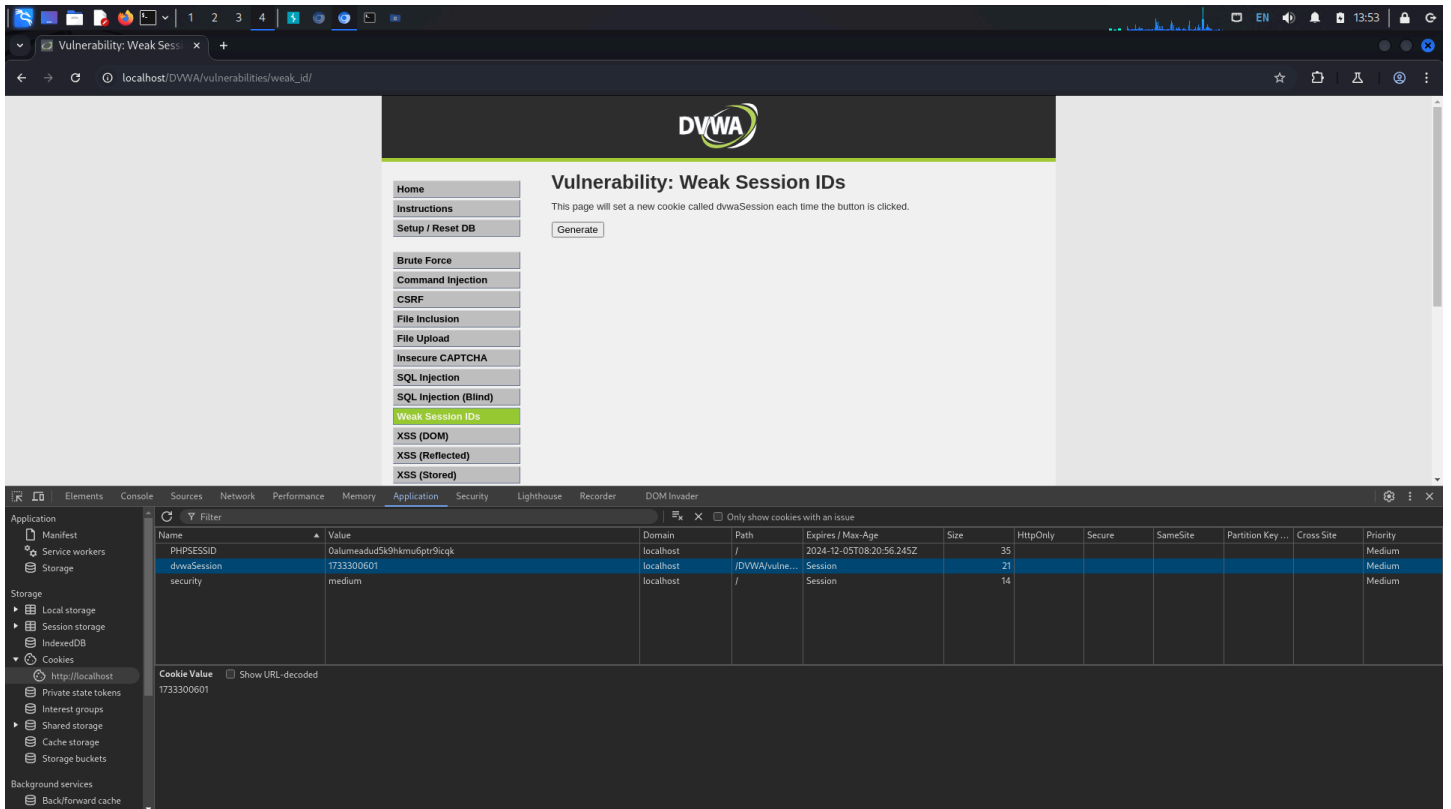
Security level was medium and PHPSESSID was "0alum...cqk"



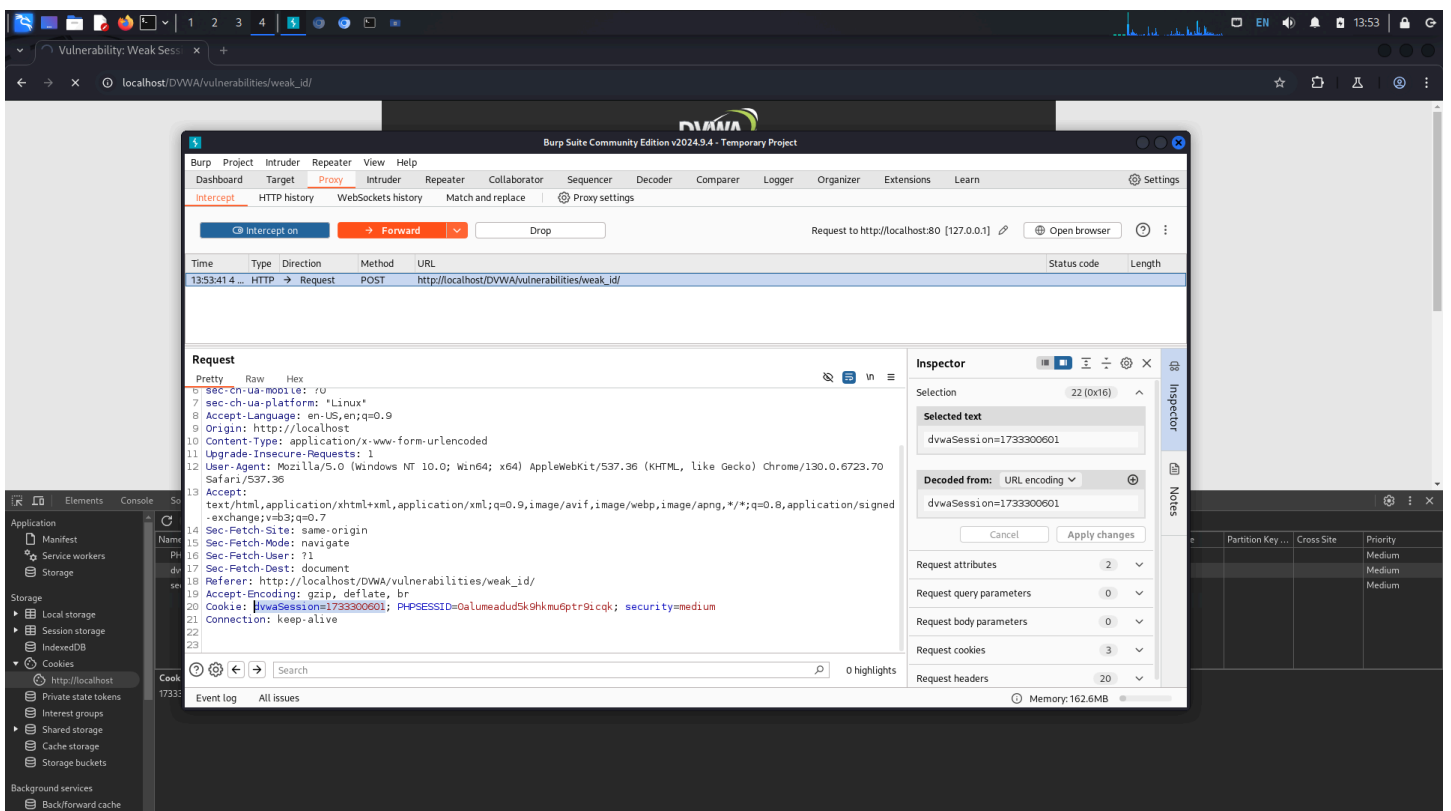
Next, I opened the developer tool to verify these cookie values after logging into the session. Upon generating session ID, the dvwaSession value was obtained as “1733300548”



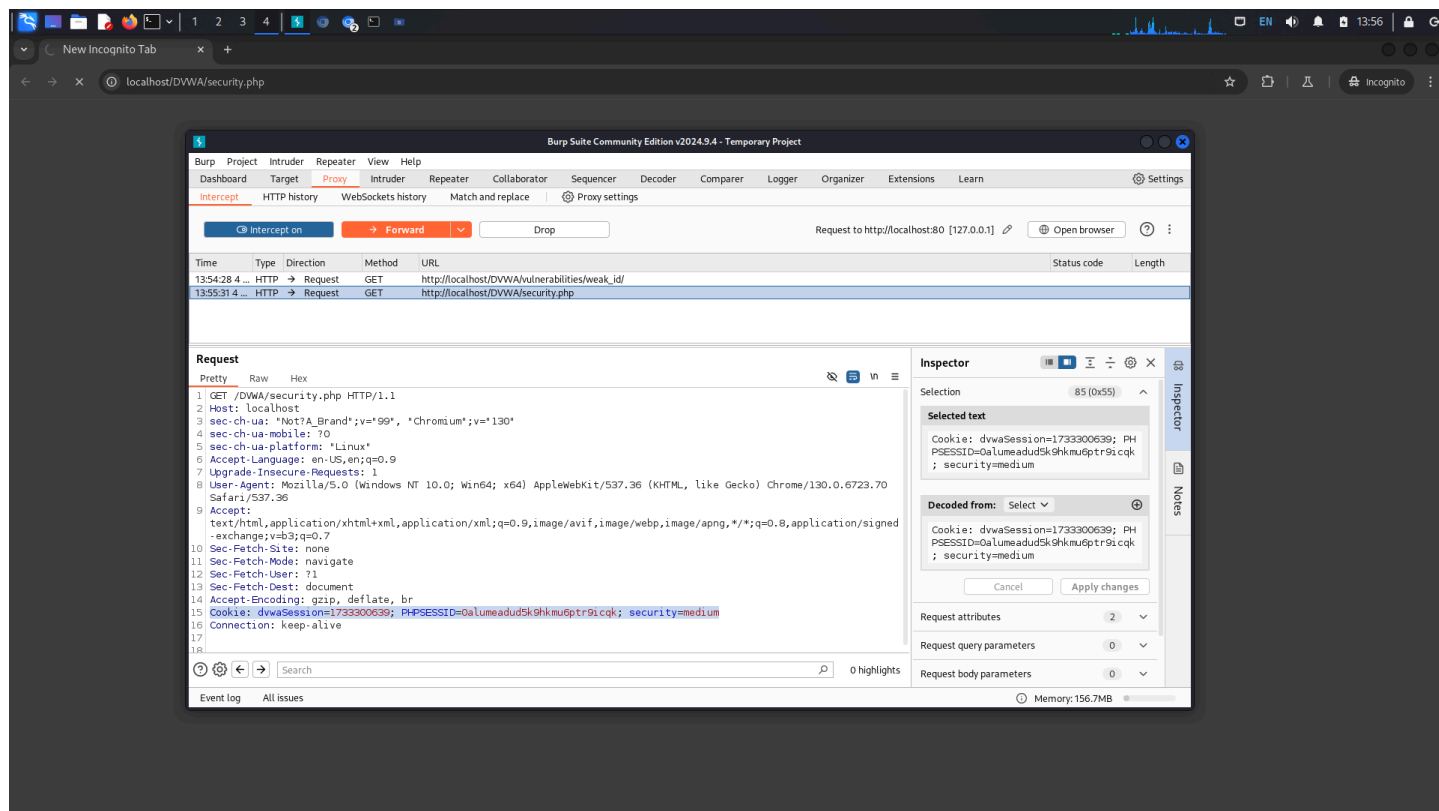
When I sent a generation request once again, I was able to see the dvwaSession value in burp suite



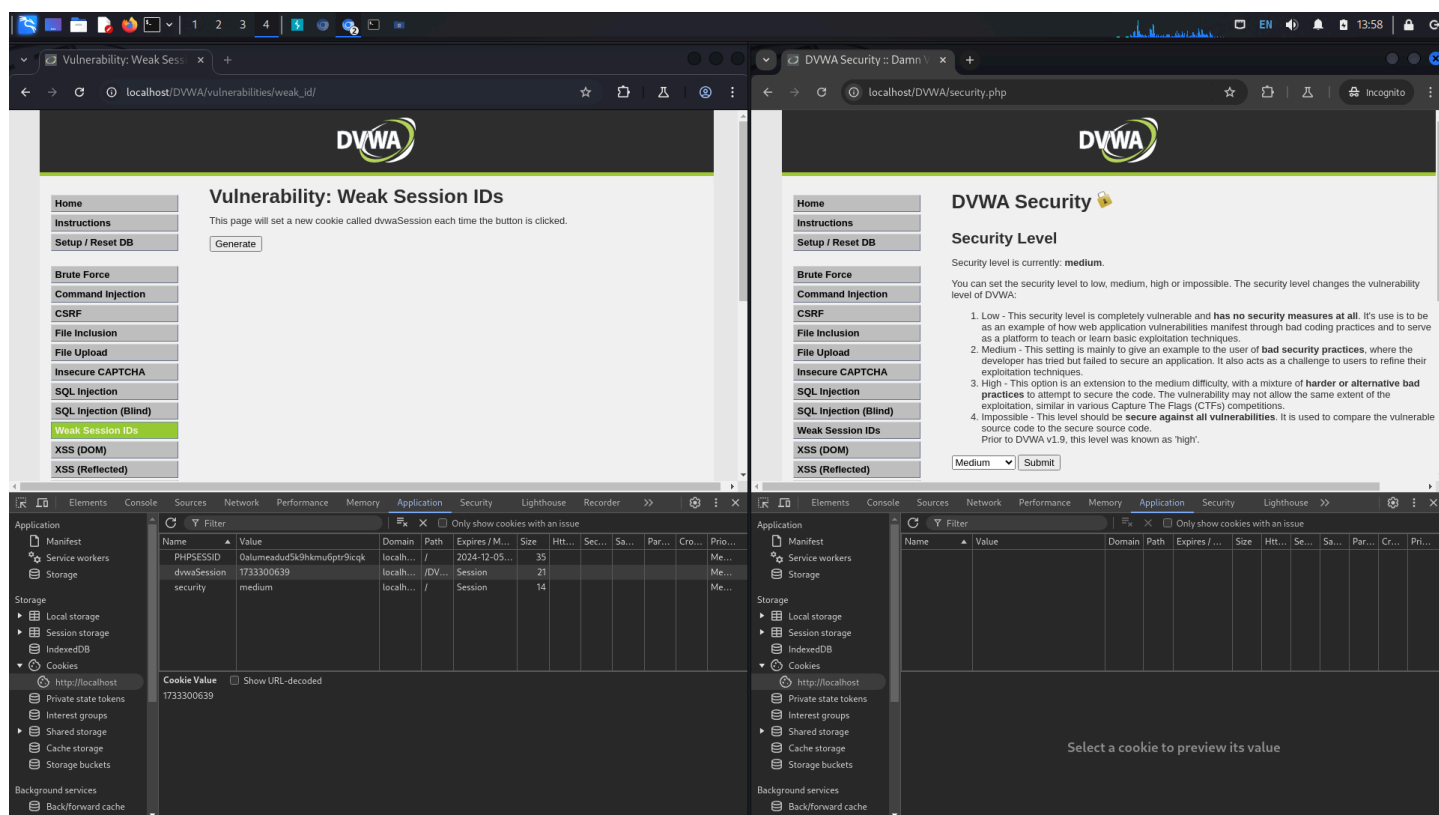
And upon forwarding the request from burp suite, I got a newly generated dvwaSession value “1733300601”
 This session ID is only the last three digits different from the previous one, “1733300548”
 Because the randomization is not so high, it would be easy for an attacker to guess or brute force and find the session ID of the login



I generated one more session ID and upon forwarding this request from burp suite, the obtained value was “1733300639”, previous session ID was “1733300601”
 This time, there was a change only in the last two digits



Now, I went on further to perform session hijacking by opening a new incognito tab (configured traffic through burp suite)
 In this tab, I requested the DVWA security page
 When the request was intercepted by Burp suite, I copy pasted the cookies line from my previous tab



Manipulating the HTTP request in Burp Suite and by copying the session cookies from my normal tab allowed me to successfully log in to the page directly without requiring to enter the login credentials in the incognito tab

In this manner, I was able to perform session hijacking utilizing burp suite and test the weak session ID vulnerability in DVWA